COS30018 - Option C - Task 4 Report

Machine Learning 1

Student Name: Kha Anh Nguyen
Student ID: 103814796
Date: September 5, 2025
Course: COS30018 - Intelligent Systems
Project: FinTech101 Stock Price Prediction System
Github repo: Leissha/stock_prediction

Table of Contents

1. Key changes:

- I built a single TFModel "factory" that constructs LSTM/GRU/RNN/BiLSTM from CLI flags.
- Added return targets (simple % or log) to reduce scale issues and focus on direction.
- Change the MinMax Scaler to Standard Z-norm feature scaling to improve boundary constraints.

2. Model-Factory Function (Design and Explanation)

- I combined all models to 1 TFModel factory class that builds **LSTM, GRU, RNN, BiLSTM** with the flexible architecture from the CLI flags (***model*** *name,* ***batch size****, number of* ***layers*** *and their* ***size, drop out rate*** *for drop out layer, and training* ***epochs****)*

```
    # Model arguments
    parser.add_argument("--model_name", type=str, default='lstm',
                        help="Model type to use",
                        choices=['lstm','gru','rnn','bilstm'])
    parser.add_argument("--layers", nargs='+', type=int, default=LAYERS,
                        help="Layer sizes separated by space (e.g: 64 32 16 128)")
    parser.add_argument("--dropout_rate", type=float, default=DROPOUT,
                        help="Dropout rate for regularization (default: 0.2)")
    parser.add_argument("--epochs", type=int, default=EPOCHS,
                        help="Number of training epochs (default: 25)")
    parser.add_argument("--batch_size", type=int, default=BATCH_SIZE,
                        help="Batch size for training (default: 32)")
```

TFModel class:

- The class will store the build config and build up the model based on the parsed arguments
- Learnt from P1, I created a frame to build all models by stacking layers based on the CLI's layers variable
  1. *For each layer size in layers:*
     - *Add* the chosen *RNN layer* with return_sequences=True except the last.
     - *Dropout* after intermediate layers for regularization (except the last)
  2. Last layer is dense layer with linear regression activation function.

  eg: layers = [50, 64, 50]

  -> number of RNN layer = len(layers) = 3 loops

  => layer 1 has 50 -> drop out layer -> layer 2 has 64 -> drop out layer -> layer 3 has 50 neurons -> dense layer

```
# Layer type mapping
_LAYER = {
    "lstm": LSTM,
    "gru": GRU,
    "rnn": SimpleRNN,
    "bilstm": LSTM
}

class TFModel:
        def __init__(self, input_size: int, model_name: str = MODEL_NAME, layers: Optional[List[int]] = LAYERS,
dropout_rate: float = DROPOUT):

        # Model configuration
        self.input_size = int(input_size)
        self.model_name = model_name.lower()
        self.layers = layers or [50, 50, 50]
        self.dropout_rate = float(dropout_rate)
```

```python
        # Determine if bidirectional
        self.bidirectional = (self.model_name == 'bilstm')

        # Build the model
        self.model = self._build_model()

    def _build_model(self):
        """Build the modelarchitecture."""
        model = Sequential(name=self.model_name)
        # Input layer flexible (n_1, n_2, features)
        from tensorflow.keras import Input  # type: ignore
        model.add(Input(shape=(None, self.input_size)))

        # Add RNN layers
        for i, units in enumerate(self.layers):
            return_sequences = i < len(self.layers) - 1

            if self.model_name == 'bilstm':
                # Bidirectional LSTM
                model.add(Bidirectional(
                    LSTM(units=units, return_sequences=return_sequences)
                ))
            else:
                # Regular RNN layers
                Layer = _LAYER[self.model_name]
                model.add(Layer(
                    units=units,
                    return_sequences=return_sequences
                ))

            # Add dropout after each layer for regularisation, option 2: + dropout and return_sequences
            if self.dropout_rate > 0 and return_sequences:
```

```
                model.add(Dropout(self.dropout_rate))

        # Final prediction layer
        model.add(Dense(units=1, activation='linear'))

        return model
# Getting simple parameters from CLI for training
    def fit(self, x_train, y_train, epochs=25, batch_size=32, learning_rate=1e-3, optimizer: str = 'adam'):
        """Train a TensorFlow model using built-in Keras features."""
        # Compile model
        optimizers = {'adam': Adam, 'rmsprop': RMSprop, 'sgd': SGD}
        opt_cls = optimizers[optimizer.lower()]
        opt = opt_cls(learning_rate=learning_rate)
        self.model.compile(optimizer=opt, loss='mean_squared_error', metrics=['mae'])

        # Train
        self.model.fit(
            x_train, y_train,
            epochs=epochs,
            batch_size=batch_size,
            verbose=1
        )

        return self

# Same predict, save, load process
      def predict_and_evaluate(self, x_test, y_test)
      def save_model(self, filepath)
      def load_model(self, filepath)
```

- Default: Constants in config file (LSTM model with 3 layers (50, 50,50), dropout rate = 0.2, batch size = 32, epochs = 50)

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, None, 50)	11,600
dropout (Dropout)	(None, None, 50)	0
lstm_1 (LSTM)	(None, None, 50)	20,200
dropout_1 (Dropout)	(None, None, 50)	0
lstm_2 (LSTM)	(None, 50)	20,200
dense (Dense)	(None, 1)	51

3. Add log/ percentage return options to target feature to improve prediction

- Before going to the result, I want to discuss the change in model prediction method here, since the current models' predictions are quite bad. I highly suspected it is because of the scaled data constraint after splitting and the wide price prediction range.
- Scaling on X_train data only helps avoid data leakage and ensure data integrity. However, it also means that the prediction scale range [0,1] is constrained to around X_train min/max range only. This caused the models to struggle with higher/lower range predictions out of the X_train (which are critical in stock as the market is spiking):

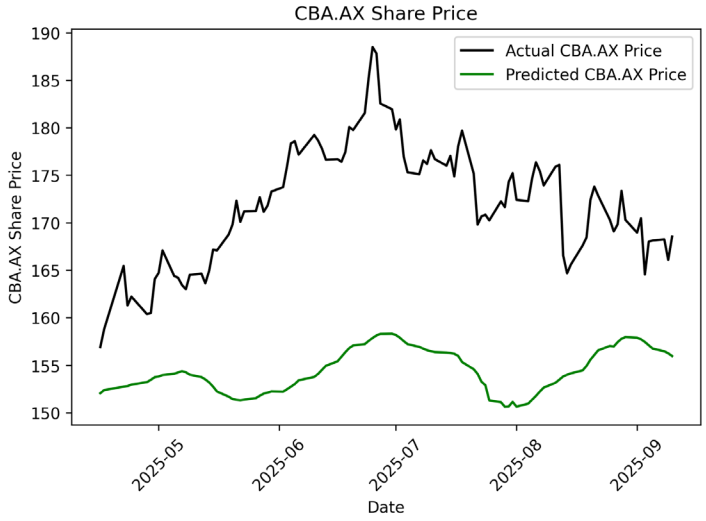


Figure 1: Default price-prediction-LSTM model with 3 layers (50, 50,50), dropout rate = 0.2, batch size = 32, ***epochs = 50*** *prediction*

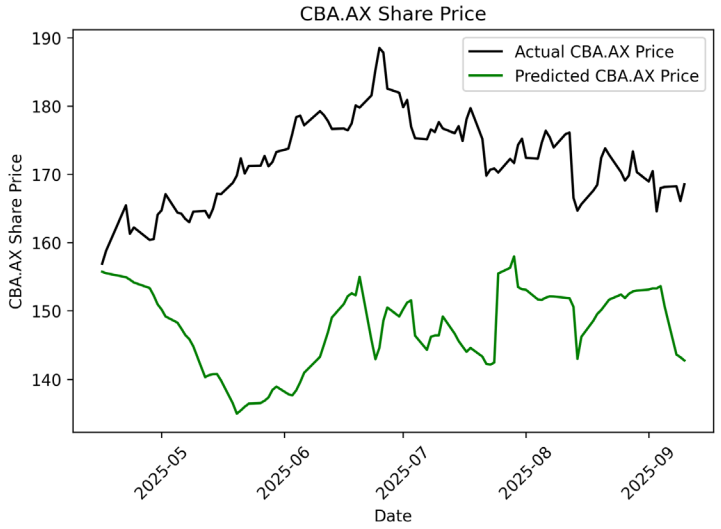


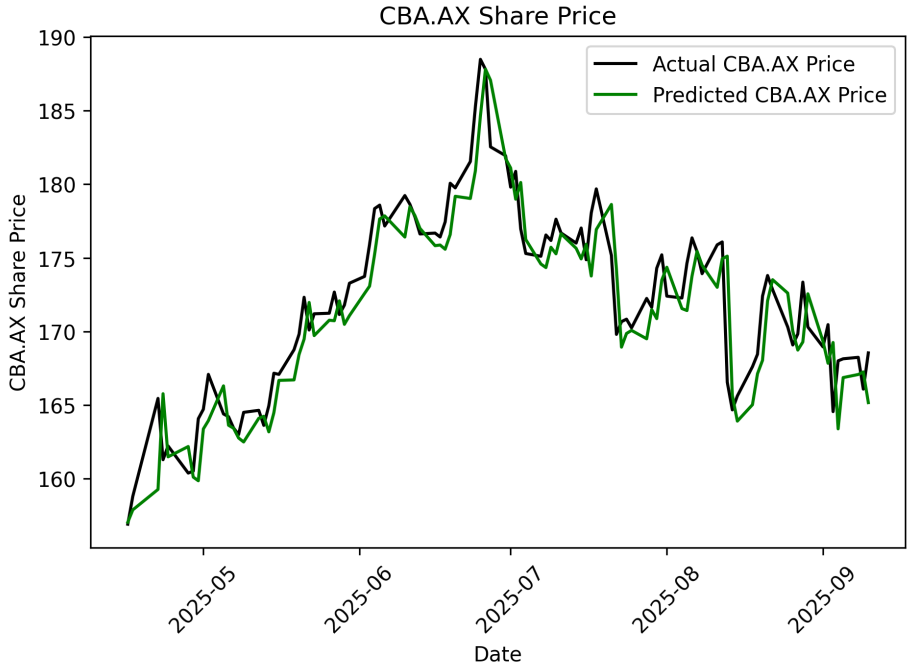
Figure 2: Default price-prediction-LSTM model with 3 layers (50, 50,50), dropout rate = 0.2, batch size = 32, ***epochs = 500*** *prediction*

- The charts show that regardless of different training hypermeters, the current model's predictions bounce within the range of the training period (1.5 years prior, below 167) and couldn't map out-of-bound prices.

- This is why I chose to add additional steps to calculate log/percentage return feature and set it as the target prediction. In addition, I changed the scaling method from MinMaxScaler to Z-norm. The prediction now will return only the changes in the current day to the next days. Which dramatically reduces the prediction ranges (0 to >160) to 0.12% mean-return (~20) and helps improve the scaled price limitations while still ensures the training integrity.

For instance:

- **OG y target** [close]: 100, 105, 102, 103, 104, 105
- **New y target** [close_return]: 0, 0.05, -0.0286, 0.0097, 0.0096, 0.0095



Log-return-prediction-LSTM model with 3 layers (50, 50,50), dropout rate = 0.2, batch size = 32, epochs = 50 prediction

Code implementation:

At main we have 3 prediction options: Default traditional price / Percentage return /Logarithm return

```
parser.add_argument("--target_ret", action="store_true", default=False,
help="Predict simple percentage returns instead of raw prices")
parser.add_argument("--log_ret", action="store_true", default=False,
help="Predict log returns instead of raw prices")
```

Recall:

- DataProcessor sequences dataframe to**:**
  - Train: create (X, y) with lookback lag_days *(e.g: 60 days)* and horizon lookup_step=1.
  - Test: append last lag_days *(e.g: 60)* rows from train to provide history;
- I added additional function to calculate return target and apply it strictly within y_pred (exclude from X to avoid data leakage).

```
class DataProcessor:
        if percent_ret or log_ret:

            Call return_feature

def return_feature(df, target_feature, use_log_returns):
    """
    https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.pct_change.html
    Original in/out features:
     training_features = ['close', 'high', 'low', 'open', 'volume']
     target_feature = 'close'

    New in/out with return feature:
```

```
    # Note that close_return not in the training set to avoid data leakage
     training_features = ['close', 'high', 'low', 'open', 'volume']
     target_feature = 'close_return'

    OG y target     [close]       : 100,   105,    102,    103,    104,    105
    Return y target [close_return]:  0,   0.05, -0.0286, 0.0097, 0.0096, 0.0095
    """
    base_col = target_feature
    return_col = f"{base_col}_return"
    if use_log_returns:
        # Log returns: r_t = log(p_t) - log(p_{t-1})
        # Use pandas Series op so .diff() is valid for type checkers
        df[return_col] = df[base_col].astype(float).apply(np.log).diff().fillna(0).values
    else:
        # Simple percentage returns: r_t = (p_t - p_{t-1}) / p_{t-1}
        # More intuitive, but asymmetric (can't lose >100% but can gain unlimited)
        df[return_col] = df[base_col].pct_change().fillna(0).values

    # Set the new target feature to the engineered return column
    target_feature = return_col

    # Validate target feature exists in the dataframe
    if target_feature not in df.columns:
        raise ValueError(f"Target feature '{target_feature}' not found in df. Available columns:
{list(df.columns)}")

    return target_feature
```

When target return option is activated, at main, I rebuild prices in point-wise: day[0] uses **last train price**; day[i] uses true price of day[i−1]. This avoids peeking and avoids error compounding across the whole horizon.

```
    # If model predicts returns, reconstruct next-day prices point-wise (no compounding)
```

```
    if percent_ret or log_ret:
        test_df = data.get('test_df', pd.DataFrame()).copy()

        base_price_col = tf_key.replace('_return', '')

        if base_price_col in test_df.columns:
            # true future prices aligned to y_test
            true_prices = test_df[base_price_col].values[-len(actual_prices):]

            # Use last available training-day price to derive first test prediction to avoid leakage
            last_training_price = float(data.get('last_training_price', test_df[base_price_col].iloc[0]))

            # Use true previous prices as base
            preds_ret = predicted_prices.reshape(-1)

            # Current prices for each step: day 0 uses seed, other days use TRUE previous prices
            current_prices = [last_training_price] + list(true_prices[:-1])

            if use_log_returns:
                # Log returns: p_t = p_{t-1} * exp(r_t)
                predicted_prices = np.array(current_prices) * np.exp(preds_ret)
            else:
                # Simple returns: p_t = p_{t-1} * (1 + r_t)
                predicted_prices = np.array(current_prices) * (1.0 + preds_ret)
            actual_prices = true_prices
        else:
            logger.warning(f"Base price column '{tf_key}' not in test_df; available sample columns:
{list(test_df.columns)[:8]}")
```

Note that, due to limited input features, I do not apply accumulative prediction (use predicted data from previous day to make prediction for the next day) for now to ensure error accumulation and concentrate on model performance first.

1. CLI

Example commands:

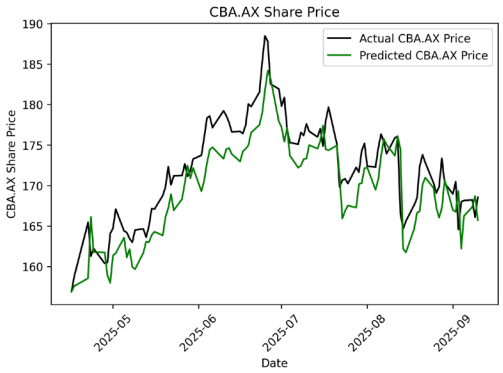
```
cd dev

# Use default settings (CBA.AX, Close price, 60 days)
# Model: LSTM, layers [50,50,50], droupout: 0.2, n_batch: 32, epochs: 50
python train.py

# Return (log), different model/layers/epochs/dropout/n_batch
python train.py --log_ret --model_name bilstm --layers 50 64 50 --epochs 60 --dropout_rate 0.4 --batch_size 64


# Return (pct), different model/layers/use default settings for the rest params
python train.py --target_ret --model_name gru --layers 50 50
```

Default setting:

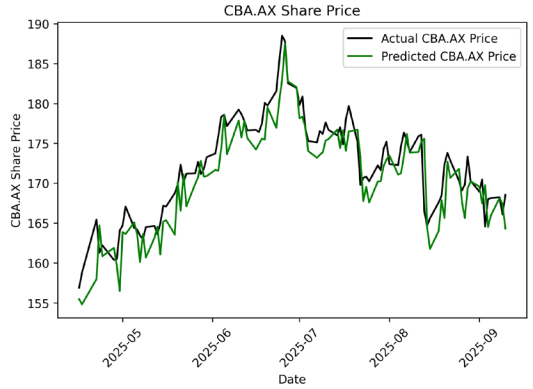
```
| `--model_name`   | str | lstm | Model type (lstm, gru, rnn, bilstm) |
| `--layers`       | list | [50, 50, 50] | Layer sizes (e.g., 64 32 16) |
| `--dropout_rate` | float | 0.2 | Dropout rate for regularization |
| `--epochs`       | int | 50 | Number of training epochs |
| `--batch_size`   | int | 32 | Batch size for training |
```

Example trials:

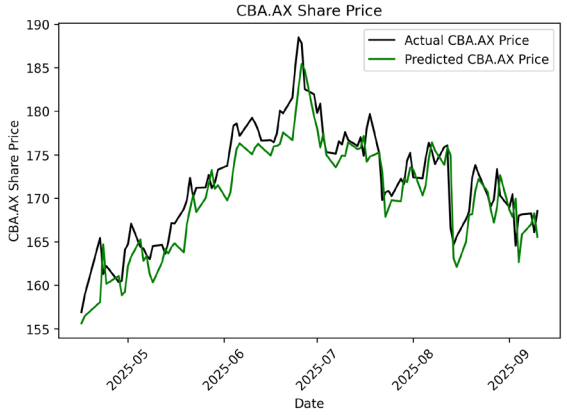
1. python train.py --log_ret --model **bilstm** --layers **50 64 50** --epochs **60**



2. python train.py --log_ret --model **rnn** --layers **64 64** --batch_size **64**



3. python train.py --**target_ret** --model **gru** --layers **50 50**



5. Experiment summary

1. Experimental Setup

- **Dataset**: CBA.AX stock data (2023-09-12 to 2025-09-11)
- **Features**: OHLCV (Open, High, Low, Close, Volume)
- **Lookback Period**: 60 days
- **Prediction Horizon**: 1 day ahead
- **Data Split**: 80% training, 20% testing (chronological)
- **Scaling**: StandardScaler applied to training features individually
- **Targets:** close (price) vs close_return (simple/log)

2. **Summary table**

Model	Architecture	Dropout	Epochs	n_batch	Target	MSE	MAE	RMSE	Directional Acc	Trading Acc	Total Profit	Profit/Trade
LSTM	[50, 50, 50]	0.2	50	32	close_return	1.32	0.87	1.15	53.9%	43.6%	$62.16	$0.80
BiLSTM	[50, 64, 50]	0.2	60	32	close_return	3.02	1.50	1.74	44.6%	37.6%	$64.70	$0.64
GRU	[50, 64]	0.3	50	32	close_return	1.89	1.06	1.37	52.0%	45.9%	$76.09	$0.78
RNN	[64, 64]	0.2	50	64	close_return	2.16	1.16	1.47	49.0%	41.7%	$74.68	$0.78
LSTM (Price)	[50, 50, 50]	0.2	50	32	close	0.84	0.88	0.92	43.1%	37.3%	$68.82	$0.67

6. Key Findings and Analysis

6.1 Return vs Price Prediction

I found out that:

- Price prediction has lower MSE/MAE as the model can learn the test range (e.g., ~120–160), so a "near-the-mean" guess keeps MSE small even when the direction is wrong.
- Return predictions have better direction accuracy (price goes up or down), which is what matters for buy/sell.

Hence, it is important to have various evaluations metrics as one bad does not mean the model is not performing well.

6.2 Architecture Insights

- **LSTM vs GRU**: GRU achieved better trading performance despite higher MSE
- **Bidirectional LSTM**: Overfit with the current limited, noisy stock data.
- **RNN**: Fast with simplest memory; suitable for this short context when we just have 60 lag days and 1 look up.

6.3 Hyperparameter Observations

- **Dropout Rate**: 0.2-0.3 range provided good regularization as the dataset is small
- **Layer Sizes**: 50-64 units per layer seemed optimal
- **Batch Size**: 32-64 range worked well
- **Epochs**: 50-60 epochs sufficient for convergence

6.4 Assumptions

- **One action per day: Buy / Sell / No-trade**. I set small (±threshold) and a simple round-trip cost to simulate real world trading,
- Action decisions is based on the previous day's true price and current model predicted price. For the first test day, model use last training price.

---

7. Conclusion

The TFModel helps ensure consistent and efficient model building. I can then have fair comparison in hyperparameters tuning and quickly find which configurations suitable in this context.